Modern GPUs: Consolidated Handout

L1: From Latency Machines to Throughput Machines
L2: Inside a Modern GPU (Hopper / Ampere)

Claude using KP's slides

Essential reading — read and understand these five

1. **Fatahalian & Houston, "A Closer Look at GPUs" (CACM 2008)** [2]. The throughput-machine argument of Lecture 1: spend area on ALUs, hide latency with many threads, share fetch/decode across SIMD lanes.
2. **Lindholm et al., "NVIDIA Tesla: A Unified Graphics and Computing Architecture" (IEEE Micro 2008)** [5]. Where SIMT, warps, the SM and the CUDA thread hierarchy were first described by the people who built them.
3. **Volkov, "Better Performance at Lower Occupancy" (GTC 2010)** [12]. Little's law applied to GPUs: what must be large is work in flight, not occupancy. The bridge from Lecture 1's occupancy to Lecture 2's asynchronous copies.
4. **Williams, Waterman & Patterson, "Roofline" (CACM 2009)** [13]. The model behind "when do GPUs win" and the 300 FLOP/B of Wall 2.
5. **Shah et al., "FlashAttention-3" (NeurIPS 2024)** [40]. One real kernel that uses TMA, warp specialisation, async `wgmma` and FP8 together, with measurements: Lecture 2 in a single paper.

1 Conventions and notation

Term	Meaning in this handout
FLOP / FLOPs	one floating-point operation / a count of them. A fused multiply–add (FMA, MAC) counts as **2 FLOPs**.
FLOP/s, TFLOPS	rate: operations per second; 1 TFLOPS = 10^{12} FLOP/s.
B, GB, TB	bytes in **decimal** units (1 GB = 10^9 B) for bandwidths and model sizes; KB for on-chip SRAM is 1024 B (vendor convention).
Lane	one SIMT execution slot; hardware analogue of one CUDA thread in a warp.
Warp	32 threads scheduled together (`warpSize` = 32 on every NVIDIA GPU to date).
Warpgroup	4 consecutive warps (128 threads); the unit of Hopper `wgmma`.
SM	Streaming Multiprocessor; the replicated core of the GPU.
CTA / block	Cooperative Thread Array = CUDA thread block.
b, s, h	batch size, sequence length, hidden (model) dimension of a transformer.
N, D	parameter count and training tokens (scaling-law notation [30]).

Pedantic note: on "cores"

NVIDIA marketing calls each FP32 lane a "CUDA core". It is not a core in the CPU sense: it has no fetch, decode or program counter of its own. The closest CPU analogue of an SM is a *multithreaded core with very wide SIMD*, and the closest analogue of a lane is one SIMD element [1, 2].

Lecture 1 — From Latency Machines to Throughput Machines

2 Where superscalar ends

2.1 The machine you know

A modern out-of-order (OoO) superscalar core spends most of its transistors on *finding and exploiting instruction-level parallelism (ILP) in a single thread*: branch prediction, register renaming, a reorder buffer (ROB), wide issue to many functional units, and a deep cache hierarchy. All of it serves one goal: reduce the **latency** of one instruction stream.

2.2 The ILP wall

Three effects ended the single-thread scaling era:

1. **Limited ILP.** Real programs expose only a handful of independent instructions per cycle; limit studies found practical ILP far below ideal machine width once realistic branch prediction and window sizes are applied [8].
2. **Super-linear cost.** Wakeup/select logic, bypass networks and the rename/ROB structures grow faster than linearly in issue width, so each extra unit of ILP costs more area and power than the last [1].
3. **End of Dennard scaling** (c. 2005). Dennard scaling [9] kept power density constant as transistors shrank; once supply voltage stopped falling, clock rates stalled and power became the binding constraint [10].

Key idea

To go faster, stop making *one* thread faster. Run *many* threads at once and judge the machine by work completed per second.

2.3 Latency versus throughput

- **Latency**: time to complete one task (seconds per task).
- **Throughput**: tasks completed per unit time (tasks per second).

A CPU minimises the latency of one stream; a GPU maximises throughput over thousands of streams. The two are linked by **Little's law** [11]:

$$\text{work in flight} \;=\; \text{throughput} \times \text{latency}. \tag{1}$$

If memory latency is fixed, the only way to raise throughput is to keep *more work in flight*. Equation (1) is the quantitative core of both lectures: occupancy (L1) and asynchronous copies (L2) are both ways to raise "work in flight".

3 The throughput idea

3.1 Hide latency, don't reduce it

A CPU *reduces* the latency of a memory access (caches, prefetchers). A GPU *hides* it: it keeps many warps resident and, whenever one stalls, issues from another that is ready. The stall still happens; it just stops costing issue slots.

3.2 Spend the transistors differently

With the same silicon budget, the GPU removes the latency-reducing machinery (OoO, speculation, large private caches) and fills the die with arithmetic units [2]. This pays off only when the program supplies enough independent work to keep those units busy.

3.3 SIMD versus SIMT

- **SIMD** (e.g. AVX-512 `vaddps zmm2, zmm0, zmm1`): one instruction operates on an explicit vector register; the lanes are visible in the ISA.
- **SIMT** (Single Instruction, Multiple Threads [5]): the programmer writes *scalar per-thread* code; hardware groups 32 threads into a warp and executes them on SIMD-like datapaths.

3.4 The warp is the unit

The scheduler issues per warp, not per thread. The 32 lanes of a warp share one fetch/decode/issue, which is where SIMT saves area and energy.

Pedantic note: "one shared program counter"

The slide says a warp has *one* PC. That was true up to Pascal. Since **Volta**, NVIDIA GPUs implement *independent thread scheduling*: each thread has its own PC and call stack, and a convergence optimiser groups threads with the same PC for issue [18]. Performance behaviour is still "one path issued at a time per warp", so the mental model in the lecture is correct for cost. It is *not* correct for synchronisation: code that relied on implicit warp-synchronous execution must now use `__syncwarp()` or the `*_sync` warp intrinsics [25].

3.5 Divergence: the cost of SIMT

When threads of a warp take different sides of a branch, the warp executes the `if` path with the `else`-lanes masked off, then the `else` path with the `if`-lanes masked, then reconverges [7, 4]. Cost $\approx$ the *sum* of the path lengths, not the maximum.

Pedantic note: "worst case 32× slower"

The 32× bound holds when all 32 lanes follow 32 *distinct* paths of equal length and the rest of the kernel is negligible. Divergence between *different* warps costs nothing extra; only intra-warp divergence serialises. Short branches are often compiled to predicated instructions (no branch at all), which also execute both sides but avoid reconvergence overhead.

4 The execution model

4.1 The hardware hierarchy

Software (CUDA)	Hardware	Rule
grid of blocks	whole GPU	blocks are distributed over SMs as capacity frees
block (CTA)	one SM	all its warps are co-resident and share shared memory
warp (32 threads)	32 lanes of one sub-partition	issued together
thread	one lane	private registers

A block never spans two SMs (thread-block *clusters* in L2 relax this only for shared memory access, not for residency).

4.2 From thread indices to warps

Threads are linearised with x fastest:

$$\text{tid} = \texttt{threadIdx.x} + \texttt{threadIdx.y} \cdot D_x + \texttt{threadIdx.z} \cdot D_x D_y,$$

and warp w holds threads $32w, \ldots, 32w + 31$ [25]. Coalescing and divergence therefore act on 32 *consecutive linear* IDs. A block whose size is not a multiple of 32 leaves the last warp partially empty; those lanes are wasted.

4.3 Occupancy

$$\text{occupancy} = \frac{\text{resident warps per SM}}{\text{maximum resident warps per SM}}.$$

Many resident warps mean one is usually ready while others wait on memory. The OoO instruction window of a CPU is replaced by a *pool of warps*.

Pedantic note: occupancy is a means, not the goal

Occupancy only matters through Little's law (1): what must be large is *independent work in flight*. A warp that issues several independent loads before using any of them contributes several units of work in flight. Volkov showed kernels at low occupancy beating high-occupancy versions by exploiting this instruction-level parallelism within each thread [12]. Raising occupancy past the point where latency is covered gains nothing and can hurt (more cache contention).

4.4 The register file

Each SM has a large register file: 65,536 32-bit registers = 256 KB on Volta through Hopper [20]. It is *partitioned* among resident threads, and a warp's context stays in registers for its whole life, so a warp switch needs no save/restore. The trade-off:

$$\text{more registers per thread} \Rightarrow \text{fewer resident warps} \Rightarrow \text{less latency hiding.}$$

Contrast a CPU, which *renames* a small architected register set onto a larger physical file to remove false dependences within one thread.

Pedantic note: allocation granularity

Registers are allocated per warp in fixed chunks (256 registers per warp on recent architectures), and per-thread counts are limited to 255. Occupancy calculators round accordingly; Design Problem L1-1 uses values that fall exactly on chunk boundaries so the rounding does not change the answer [25].

4.5 The memory hierarchy

Registers (per thread) → L1 / shared memory (per SM) → L2 (whole GPU) → HBM (global). GPUs lead with **bandwidth** (TB/s of HBM) rather than low latency; L2 has hundreds of cycles of latency and HBM roughly 500 or more (numbers in Table 1). Think "bytes per second delivered to the ALUs", not "cycles to one load".

4.6 Memory placement and the on-chip network

Jin et al. measured the network-on-chip of a V100 [15]: an SM's L2 access latency depends on the physical distance between that SM and the L2 slice, from about 183 cycles for near SMs to over 215 cycles for far ones (about 17% spread). Throughput design tolerates this because (i) aggregate NoC bandwidth is roughly uniform, and (ii) thousands of resident warps cover the variance. Latency is hidden, not reduced.

4.7 Coalescing

The load/store unit merges the 32 addresses of a warp's memory instruction into the fewest aligned **32-byte sectors** (cache lines are 128 B = 4 sectors) [25]. The *same* instruction `ld a[idx]` can cost 1 line or 32, depending only on which addresses the 32 lanes touch together. Section 8.3 quantifies this.

4.8 Shared memory

Shared memory is on-chip SRAM per SM, about as fast as L1, but *software-managed*: the program decides what lives there and when (a scratchpad). **Tiling** loads a block of data once from HBM and reuses it many times from shared memory, turning a bandwidth-bound kernel into a compute-bound one [3]. It is the largest single lever in kernel tuning.

Pedantic note: bank conflicts

Shared memory is split into 32 banks of 4-byte words. If two lanes of a warp access different words in the same bank, the accesses serialise (a bank conflict). Padding arrays (e.g. `tile[32][33]`) or swizzled layouts avoid this. Hopper's TMA applies such swizzles in hardware (Section 10.3).

5 Programming the GPU (minimal CUDA)

5.1 The compiler: `nvcc`

One `.cu` file holds host and device code. `nvcc` splits it: host code goes to the system C++ compiler; device code is compiled to **PTX** (a stable virtual ISA [26]) and then to **SASS** (the real, per-architecture machine code). A *fatbin* bundles SASS for several GPUs plus PTX, which the driver JIT-compiles for GPUs newer than the build.

5.2 The runtime: launching work

```
cudaMalloc(&dx, n*sizeof(float));
cudaMemcpy(dx, x, n*sizeof(float), cudaMemcpyHostToDevice);
int B = 256, G = (n + B - 1) / B; // ceil(n/B) blocks
saxpy<<<G, B>>>(n, a, dx, dy); // asynchronous launch
cudaMemcpy(y, dy, n*sizeof(float), cudaMemcpyDeviceToHost); // waits
```

`«<G,B»>` launches G blocks of B threads. The launch returns immediately; the work is queued on a *stream*. A plain `cudaMemcpy` on the default stream waits for prior work, which is why the final copy acts as a synchronisation point.

5.3 SAXPY end to end

```
__global__ void saxpy(int n, float a, float *x, float *y) {
  int i = blockIdx.x * blockDim.x + threadIdx.x; // global id
  if (i < n) y[i] = a * x[i] + y[i]; // guard the tail
}
```

The loop body becomes one thread; threads 0–31 form warp 0; adjacent `i` touch adjacent `x[i]`, so every load and store is fully coalesced.

Pedantic note: the PTX on the slide is illustrative

The slide shows separate `mul.f32` and `add.f32`. By default `nvcc` contracts $a{\cdot}x{+}y$ into a single `fma.rn.f32` (flag `-fmad=true`), which also rounds once instead of twice. Real PTX also contains the address arithmetic, the bounds check and a parameter load for `a`. SAXPY does 2 FLOPs per 12 bytes moved (load x, load y, store y), an arithmetic intensity of 1/6 FLOP/B, so it is *memory-bound* on every GPU ever built.

6 Synthesis

6.1 The CPU ↔ GPU dictionary (L1 version)

Superscalar CPU	Throughput GPU
OoO instruction window	pool of resident warps (occupancy)
branch prediction	SIMT execution + divergence masking
register renaming	huge register file, statically partitioned
caches for reuse (automatic)	coalescing + shared-memory scratchpad
superscalar issue width	warp width (32 lanes)
speed up *one* thread	overlap *thousands* of threads
reduce latency	hide latency

Lecture 2 extends this table (Section 12.3).

6.2 When do GPUs win? The roofline

The roofline model [13] bounds attainable throughput:

$$P_{\text{attainable}} = \min(P_{\text{peak}},\ B \cdot I), \qquad I = \frac{\text{FLOPs}}{\text{bytes moved from memory}}. \tag{2}$$

The *ridge point* $I^* = P_{\text{peak}}/B$ separates memory-bound kernels ($I < I^*$) from compute-bound ones. GPUs win with lots of independent work, regular (coalesced) access, little divergence and enough arithmetic per byte. They struggle with pointer chasing, heavy branching, low parallelism and long serial dependence chains.

Lecture 2 — Inside a Modern GPU (Hopper / Ampere)

7 Motivation: the three walls

7.1 The reference job: training Llama 3 70B

- $N = 70 \times 10^9$ parameters, $D = 15 \times 10^{12}$ tokens [31, 32].
- Training compute $C \approx 6ND$ [30]: 2 FLOPs per parameter per token in the forward pass, 4 in the backward pass.

$$C \approx 6 \times 70 \times 10^9 \times 15 \times 10^{12} = 6.3 \times 10^{24}\,\text{FLOP}.$$

- Reported cost: 6.4×10^6 H100 GPU-hours [31].

Derived quantities.

$$\begin{aligned}
\text{GPU-seconds} &= 6.4 \times 10^6 \times 3600 = 2.304 \times 10^{10}\,\text{s},\\
\text{sustained per-GPU rate} &= \frac{6.3 \times 10^{24}}{2.304 \times 10^{10}} \approx 2.73 \times 10^{14}\,\text{FLOP/s} = 273\,\text{TFLOPS},\\
\text{model FLOPs utilisation (MFU)} &= 273/989 \approx 28\%,\\
\text{GPU-years} &= 6.4 \times 10^6/8760 \approx 730.
\end{aligned}$$

Pedantic note: what the 28% means

The GPU-hour figure in the model card covers the whole training effort as reported, which may include restarts and overheads, so the derived 28% is a lower bound on the MFU of the steady-state run. The Llama 3 paper reports 38–43% BF16 MFU for the 405B model on 16K H100s [32]. The "$6ND$" rule also ignores attention FLOPs (the $4bs^2h$ term below), which are small at these sizes but not zero.

7.2 The three walls

Run the same job on the **L1 machine**: FP32 SIMT lanes only, perfect occupancy and coalescing.

1. **Wall 1 — Compute.** H100 FP32 SIMT peak is 67 TFLOPS; its BF16 Tensor Core peak is 989 TFLOPS [21]. Ratio $989/67 \approx 14.8$, so at equal utilisation the job takes $\approx 15\times$ longer: $730 \times 14.8 \approx 10{,}800$ GPU-years.
2. **Wall 2 — Feeding.** 989 TFLOPS / 3.35 TB/s $\approx$ 295 FLOP/B (the slides round to 300). Every byte fetched from HBM must be used in about 300 FLOPs or the Tensor Cores starve.
3. **Wall 3 — Size.** Mixed-precision Adam training holds ≈ 16 B of state per parameter (below): $70 \times 10^9 \times 16 = 1.12$ TB, against 80 GB of HBM per GPU. Weights alone in BF16 are 140 GB.

Where 16 bytes per parameter comes from. For mixed-precision training with Adam [34, 36]:

State	Bytes/param
BF16/FP16 weights (used in GEMMs)	2
BF16/FP16 gradients	2
FP32 master copy of weights	4
Adam first moment m (FP32)	4
Adam second moment v (FP32)	4
Total	**16**

Pedantic note: activations are extra

The 16 B/param figure is *model state only*. Activations saved for the backward pass scale with $b{\cdot}s{\cdot}h{\cdot}$(layers), not with N, and come on top. Dividing 1.12 TB by 80 GB gives 14 GPUs as a floor for state alone; in practice the state is sharded (ZeRO/FSDP [34]) and combined with tensor and pipeline parallelism across thousands of GPUs.

8 The modern SM up close

8.1 Anatomy of an H100 SM

An SM has **4 sub-partitions** (processing blocks). Each has [20]:

- one warp scheduler + dispatch unit (one instruction per cycle);
- **32** FP32 lanes, 16 INT32 lanes, 16 FP64 lanes;
- **one** fourth-generation Tensor Core;
- a 16,384 × 32-bit register file slice (64 KB);
- load/store units and an SFU (transcendentals).

Shared by the SM: a combined **256 KB** L1 data cache / shared memory (up to 228 KB configurable as shared memory) and the TMA unit.

H100 SXM5, per GPU	**Value**
SMs	132
FP32 lanes per SM (total)	128 (16,896)
Tensor Cores per SM (total)	4 (528)
Register file per SM	256 KB
L1 + shared per SM	256 KB (shared ≤ 228 KB)
L2 cache	50 MB
HBM3	80 GB at 3.35 TB/s
Max resident threads / warps / blocks per SM	2048 / 64 / 32
NVLink 4	18 links, 900 GB/s total (450 GB/s each direction)

Pedantic note: lanes per sub-partition

"128 FP32 lanes" is per *SM*; each sub-partition has 32. A warp's FP32 instruction therefore occupies its sub-partition's 32 lanes for one cycle. The NVLink figure is often misquoted: 18 links × 50 GB/s (bidirectional) = 900 GB/s total, i.e. 450 GB/s per direction [20], which is ≈7× PCIe 5 x16 per direction,

not "2× HBM".

8.2 Scheduling, concretely

Each of the 4 schedulers picks one *ready* warp per cycle and issues one instruction. A stalled warp is simply not picked; switching costs nothing because every warp's registers are resident. There is no ROB and no speculation: warps replace the OoO engine.

Toy example (slide). One scheduler, 4 warps, each issues 3 instructions (the third a load) and then waits 18 cycles. Cycles 0–11 issue 12 instructions; warp 0's load returns at cycle 20, so cycles 12–19 are **8 idle issue slots**.

How many warps hide one HBM load? By Little's law, with latency ≈ 600 cycles and about 10 independent instructions per warp between long-latency loads:

$$\text{warps needed} \approx \frac{600}{10} = 60 \text{ per scheduler},$$

but H100 holds at most $64/4 = 16$ warps per scheduler. So each warp must keep **several loads in flight** at once, which is exactly Wall 2.

Profiler vocabulary (Nsight Compute). *Selected*: issued this cycle. *Not Selected*: ready but another warp won. *Stall Long Scoreboard*: waiting on global/local memory. *Stall Barrier*: waiting at `__syncthreads`. *Stall Math Pipe Throttle*: the ALUs are saturated (the good kind of stall).

Pedantic note: scheduling policy

NVIDIA does not document the warp-selection policy. Academic models use loose round-robin or greedy-then-oldest (GTO) [4]; microbenchmarks suggest variants of these. "No dual issue since Volta" refers to issuing two instructions from the same warp in one cycle; Kepler and Maxwell could.

8.3 The memory hierarchy with numbers

Level	Latency	Bandwidth	Size / scope
Registers	∼1 cycle (operand)	∼100s TB/s aggregate	256 KB/SM
L1 / shared	∼30–35 cycles	∼10s TB/s aggregate	≤256 KB/SM
L2	∼200–270 cycles	∼5–10 TB/s	50 MB, all SMs
HBM3	∼470–660 cycles	3.35 TB/s	80 GB

Table 1: H100 memory hierarchy. Latencies are microbenchmark results [17, 16] and depend on access pattern, clock and TLB state; bandwidths for registers, L1 and L2 are order-of-magnitude aggregates, not vendor specifications.

Sectors and coalescing. One warp loading 4-byte floats:

Pattern	Bytes requested	Sectors touched	Efficiency
`A[i]`	128	4	100%
`A[2*i]`	128	8	50%
`A[idx[i]]` (random)	128	up to 32	down to 12.5%

Who manages each level? Registers: the compiler (`-maxrregcount`, `__launch_bounds__`, `setmaxnreg`). Shared memory: the programmer. L1: hardware, with per-load cache hints. L2: hardware, plus an optional *persistence window* (Ampere+). HBM: the programmer (allocation, layout, prefetch).

L2 persistence (Ampere+).

```
cudaDeviceSetLimit(cudaLimitPersistingL2CacheSize, 16 << 20);
cudaStreamAttrValue a = {};
auto &w = a.accessPolicyWindow;
w.base_ptr = table; // hot data
w.num_bytes = 16 << 20; // 16 MB window
w.hitRatio = 1.0f;
w.hitProp = cudaAccessPropertyPersisting;
w.missProp = cudaAccessPropertyStreaming;
cudaStreamSetAttribute(s, cudaStreamAttributeAccessPolicyWindow, &a);
```

Accesses in the window are preferentially retained in the set-aside part of L2, so a small hot table (embedding, lookup) is not evicted by streaming traffic [25].

9 Wall 1: compute — Tensor Cores and mixed precision

9.1 Where the FLOPs go in a transformer

For one GPT-style layer (multi-head attention, $4h$ GELU feed-forward), forward pass, counting 2 FLOPs per MAC [33]:

Operation	Shape	FLOPs
Q, K, V projections	3 GEMMs $[bs \times h] \cdot [h \times h]$	$6bsh^2$
$QK^\top$ and softmax$(QK^\top)V$	2 batched matmuls	$4bs^2h$
Output projection	$[bs \times h] \cdot [h \times h]$	$2bsh^2$
FFN up	$[bs \times h] \cdot [h \times 4h]$	$8bsh^2$
FFN down	$[bs \times 4h] \cdot [4h \times h]$	$8bsh^2$
GELU, LayerNorm, residuals, softmax	elementwise	$O(bsh)$, $O(bs^2)$
Total		$24bsh^2 + 4bs^2h$

The dense-to-attention ratio is $24bsh^2/4bs^2h = 6h/s$. With $h = s = 8192$ that is 6:1, i.e. 86% dense GEMM and 14% attention matmul; elementwise work is negligible in FLOPs. Backward costs $\approx 2\times$ forward with the same mix.

Key idea

Essentially every FLOP is a matrix multiply. Build hardware for GEMM, not for the scalar FMA. Elementwise operations are cheap in FLOPs but not in *bytes*, which is Wall 2.

Pedantic note: Llama 3 is not exactly this layer

Llama 3 70B uses grouped-query attention (8 KV heads, so the K, V projections are 8× smaller) and a SwiGLU FFN with three matrices of inner size 28,672 $\approx 3.5h$ [32]. Its per-layer count therefore differs from $24bsh^2$, but the conclusion (dense GEMMs dominate) is unchanged. The slide's phrase "≈ 24 matrix multiplies" should read "$24bsh^2$ FLOPs of dense GEMM": a layer has 6 weight GEMMs and 2 attention matmuls.

9.2 All the L1 tricks, measured

Boehm's FP32 SGEMM worklog (4096^2, RTX A6000) [35]:

Kernel	GFLOPS	% of cuBLAS
Naive	309	1.3
Global-memory coalescing	1,987	8.5
Shared-memory tiling	2,980	12.8
1D register (block) tiling	8,475	36.5
2D register (block) tiling	15,972	68.7
Vectorised loads (`float4`)	18,237	78.4
Warp tiling	21,779	93.7
cuBLAS	23,250	100

Every L1 idea works: 23,250/309 $\approx 75\times$. Yet cuBLAS stops at 23.25/38.7 $\approx 60\%$ of the A6000's FP32 peak, and that ceiling is itself the problem. The same card's FP16 Tensor Core peak is ≈ 155 TFLOPS dense (≈ 310 with sparsity) [22].

9.3 One instruction, 64× the math

SIMT inner loop: one warp `FFMA` = 32 MACs. One warp-level Tensor Core instruction [26]

```
mma.sync.aligned.m16n8k16.row.col.f32.f16.f16.f32
```

computes

$$D_{16\times8} = A_{16\times16}\,B_{16\times8} + C_{16\times8}, \qquad 16 \cdot 8 \cdot 16 = 2048 \text{ MACs.}$$

Same issue slot, $2048/32 = 64\times$ the MACs. Each thread supplies 4 registers of A (8 FP16 values), 2 of B (4 FP16), and 4 FP32 values of C/D. Operands are reused *inside* the unit, not re-read from the register file: the scalar ceiling was issue bandwidth and register-file bandwidth, not ALU count.

Pedantic note: 64× per instruction is not 64× per second

A Tensor Core instruction occupies its unit for several cycles, so "64× the math per issue slot" does not translate to 64× throughput. The chip-level ratio is what counts: H100 dense FP16 Tensor (989 TFLOPS) over FP32 SIMT (67 TFLOPS) is $\approx 14.8\times$, the Wall-1 factor. (On H100 each SM sustains 1024 dense FP16 MACs per cycle on Tensor Cores versus 128 FP32 FMAs on the lanes, quoted at different boost clocks [20].)

9.4 Tensor Cores in real code: WMMA

```
#include <mma.h>
using namespace nvcuda;
// one warp -> one 16x16 tile of C = A (row-major, MxK) * B (col-major, KxN)
```

```
__global__ void wmma_gemm(const half *A, const half *B, float *C,
                          int M, int N, int K) {
  int wm = blockIdx.y * blockDim.y + threadIdx.y; // tile row
  int wn = (blockIdx.x * blockDim.x + threadIdx.x) / 32; // tile col
  wmma::fragment<wmma::matrix_a, 16,16,16, half, wmma::row_major> a;
  wmma::fragment<wmma::matrix_b, 16,16,16, half, wmma::col_major> b;
  wmma::fragment<wmma::accumulator, 16,16,16, float> c;
  wmma::fill_fragment(c, 0.0f);
  for (int k = 0; k < K; k += 16) {
    wmma::load_matrix_sync(a, A + wm*16*K + k, K);
    wmma::load_matrix_sync(b, B + wn*16*K + k, K);
    wmma::mma_sync(c, a, b, c); // c += a*b
  }
  wmma::store_matrix_sync(C + wm*16*N + wn*16, c, N, wmma::mem_row_major);
}
```

There is no per-element loop: a *fragment* is opaque, and the hardware decides which lane holds which element.

Pedantic note: what this kernel assumes

It assumes M, N, K are multiples of 16, `blockDim.x` is a multiple of 32, and the grid exactly covers C; there are no bounds checks. It reads A and B straight from global memory with no shared-memory staging, so it is a correctness illustration, not a fast GEMM. Hopper's `wgmma` (below) is warpgroup-wide, asynchronous, and reads operands from shared memory.

9.5 Number formats

Format	Sign	Exp.	Mant.	Max finite	Notes
FP32	1	8	23	3.4×10^{38}	IEEE single
TF32	1	8	10	$\approx 3.4 \times 10^{38}$	Tensor Core input only (stored as 32 bits)
FP16	1	5	10	65,504	IEEE half
BF16	1	8	7	$\approx 3.4 \times 10^{38}$	FP32 range, low precision [37]
FP8 E4M3	1	4	3	448	no ∞; forward pass [38]
FP8 E5M2	1	5	2	57,344	IEEE-like; gradients [38]

H100 SXM dense throughput ladder [21]: FP64 Tensor 67, TF32 495, FP16/BF16 989, FP8 1979 TFLOPS; FP32 SIMT 67. Each step down roughly doubles throughput and halves bytes. 2:4 structured sparsity doubles the quoted numbers again.

9.6 Why FP8? Serving a 70B model

Decoding one token (batch 1) reads every weight once, so

$$t_{\text{token}} \geq \frac{\text{weight bytes}}{\text{HBM bandwidth}}.$$

- BF16: $70 \times 10^9 \times 2 = 140\,\text{GB} > 80\,\text{GB}$, so at least 2 GPUs.
- FP8: 70 GB, fits one GPU; $t \geq 70/3350\,\text{s} \approx 21\,\text{ms}$ (≤ 48 tokens/s).

FP8 halves the bytes (decode, bandwidth-bound) and doubles Tensor Core FLOPs (prefill and training, compute-bound). In CUDA the types are `__nv_fp8_e4m3` and `__nv_fp8_e5m2` (`cuda_fp8.h`); in practice NVIDIA's Transformer Engine (`te.fp8_autocast`) manages scaling [29].

Pedantic note: the 21 ms bound

It ignores the KV cache (which also must be read every token and grows with context), activations, and kernel launch overheads, and assumes the full 3.35 TB/s is achieved (80–90% is typical). With batch > 1 the weights are reused across requests, raising arithmetic intensity, which is why serving systems batch.

9.7 Why FP8 needs scaling

E4M3's range is $[2^{-9}, 448]$ including subnormals: about 18 binades. Real tensors would overflow or underflow to zero. A per-tensor scale s maps them into range:

$$q = \mathrm{round}_{\mathrm{FP8}}(x \cdot s), \qquad \hat{x} = q/s, \qquad s = \frac{\mathrm{FP8}_{\max}}{\mathrm{amax}(x)} \cdot 2^{-\mathrm{margin}}.$$

Transformer Engine's *delayed scaling* uses the amax history of previous iterations to choose s [29]. Accumulation stays in FP32 (or higher internal precision).

10 Wall 2: feeding — data movement

10.1 Feeding the beast

Compute became $\approx 15\times$ faster (Wall 1). If data delivery does not keep up, the Tensor Cores starve. For a GEMM tile of size $m \times n$ with depth k, the work is $2mnk$ FLOPs and the operand traffic is $(mk + kn)$ elements, so

$$I_{\mathrm{tile}} = \frac{2mnk}{(mk + kn) \cdot w} = \frac{2mn}{(m+n)\, w} \ \mathrm{FLOP/B}$$

for w bytes per element. For square tiles $m = n = T$ this is T/w: in BF16 ($w = 2$), reaching 295 FLOP/B from HBM needs $T \gtrsim 600$. That is why tiles are large, why they are reused through L2 and shared memory, and why shared memory (228 KB) must be refilled continuously and *predictably*.

Pedantic note: intensity of a GEMM

The slide-era formula $\frac{nk}{n+k}$ drops the factor of 2 (FLOPs per MAC) and the element width. Equation above is the complete form. It counts loads of A and B only; the C tile is read/written once per tile and amortised over k.

The four bottlenecks that follow appeared in this order, and each forced a hardware feature.

10.2 Bottleneck 1: loads held registers hostage → `cp.async` (Ampere)

Before Ampere a tile went HBM → register (`LDG`) → shared memory (`STS`). The destination register is reserved for the whole load latency (∼500 cycles), so registers are spent on data in flight rather than on math, and occupancy or pipeline depth suffers. Ampere's `cp.async` (SASS `LDGSTS`) copies global → shared directly, bypassing registers [19, 25].

```
// issue tile k+S-1 while earlier tiles are in flight (cuda_pipeline.h)
__pipeline_memcpy_async(&As[stage][s], &A[g], 16);
__pipeline_commit(); // close this batch
__pipeline_wait_prior(STAGES - 2); // tile k has landed
__syncthreads();
compute(As[k % STAGES]);
```

Consequences: fewer registers per thread, so more warps or larger accumulators; and 3–4 stages in flight instead of 2. The Nsight symptom before the change is *Stall Long Scoreboard*.

The same idea with `cuda::pipeline`.

```
auto block = cooperative_groups::this_thread_block();
__shared__ cuda::pipeline_shared_state<cuda::thread_scope_block, 2> state;
auto pipe = cuda::make_pipeline(block, &state);
for (int k = 0, f = 0; k < T; ++k) {
  for (; f < T && f < k + 2; ++f) { // keep 2 tiles in flight
    pipe.producer_acquire();
    cuda::memcpy_async(block, sh[f % 2], &A[f * TILE],
                       cuda::aligned_size_t<16>(BYTES), pipe);
    pipe.producer_commit();
  }
  pipe.consumer_wait(); // tile k ready
  compute(sh[k % 2]); // ...while k+1 loads
  pipe.consumer_release();
}
```

Computing tile k overlaps loading tile $k{+}1$ (Design Problem L2-1 quantifies the gain).

10.3 Bottleneck 2: address math ate the issue slots → TMA (Hopper)

With `cp.async` every thread still computes addresses, checks bounds and applies the swizzle for its share of the tile: for a 128-thread block, tens of integer instructions per thread that compete with MMAs for issue slots. Hopper's **Tensor Memory Accelerator** moves a whole multidimensional tile from one instruction issued by a single thread [20, 25].

```
// Host: describe the tensor once
CUtensorMap tmap;
cuTensorMapEncodeTiled(&tmap, CU_TENSOR_MAP_DATA_TYPE_FLOAT16,
    2, A, dims, strides, box, elemStrides,
    CU_TENSOR_MAP_INTERLEAVE_NONE, CU_TENSOR_MAP_SWIZZLE_128B,
    CU_TENSOR_MAP_L2_PROMOTION_NONE, CU_TENSOR_MAP_FLOAT_OOB_FILL_NONE);

// Device: one thread moves the whole tile
__shared__ alignas(128) half sA[128][64];
__shared__ barrier bar; // cuda::barrier (init omitted)
if (threadIdx.x == 0) {
  cde::cp_async_bulk_tensor_2d_global_to_shared(&sA, &tmap, x, y, bar);
  token = cuda::device::barrier_arrive_tx(bar, 1, sizeof(sA)); // expect bytes
} else {
  token = bar.arrive();
}
bar.wait(std::move(token)); // tile is in smem
```

The descriptor holds shape, strides, box (tile) size and swizzle; address generation, out-of-bounds fill and bank-conflict-free swizzling are done in hardware. Completion is signalled through a transaction-count `mbarrier`: the barrier completes when the expected number of bytes has arrived.

10.4 Bottleneck 3: synchronous MMA → async `wgmma` + warp specialisation (Hopper)

`mma.sync` blocks its warp until the result is in registers, so the same warp cannot also be issuing loads. Hopper's `wgmma.mma_async` lets a **warpgroup** (128 threads) launch an $m64\,nN\,k16$ MMA (N up to 256) that reads A and B directly from shared memory and returns immediately [26]. Kernels then use **warp specialisation**:

```
// sketch, CUTLASS / FlashAttention-3 style
```

```
if (warpgroup == 0) { // PRODUCER
  asm volatile("setmaxnreg.dec.sync.aligned.u32 40;");
  for (k...) { wait(empty[s]); tma_load(A_k, B_k, smem[s], full[s]); }
} else { // CONSUMERS
  asm volatile("setmaxnreg.inc.sync.aligned.u32 232;");
  for (k...) { wait(full[s]);
    wgmma_m64n256k16(acc, smem[s]); // returns at once
    wgmma_wait_group<1>(); // keep one MMA in flight
    arrive(empty[s]); }
}
```

The producer issues TMA loads into a shared-memory ring buffer; consumers run `wgmma` on full slots and release them. `setmaxnreg` moves registers from the producer (which needs few) to the consumers (which hold large accumulators). This is the GPU form of Smith's *decoupled access/execute* architecture [14].

Pedantic note: register counts

40 and 232 are typical CUTLASS Hopper values, not architectural constants. `setmaxnreg` takes a multiple of 8 in $[24, 256]$, and the per-SM total must still fit in 64K registers [26, 27].

The evidence: FlashAttention-2 versus -3. Same algorithm on one H100: FlashAttention-2 reaches ≈ 350 TFLOPS FP16 (about 35% of peak); FlashAttention-3, using TMA, warp specialisation, async `wgmma` with "ping-pong" overlap of softmax and GEMM, reaches ≈ 740 TFLOPS FP16 (about 75%) and ≈ 1.2 PFLOPS in FP8 [39, 40]. The features are not a checklist; together they double the fraction of the chip a real kernel uses.

10.5 Bottleneck 4: the same tile fetched twice → thread-block clusters (Hopper)

In a GEMM, blocks in the same row of C need the same A tile and blocks in the same column need the same B tile. Independent blocks on different SMs each fetch it from L2. A **cluster** is a group of blocks guaranteed to be co-scheduled on SMs of the same GPC [20, 25]. Within a cluster:

- **TMA multicast**: one L2 read delivered to the shared memory of several SMs;
- **distributed shared memory (DSMEM)**: a block can load, store and perform atomics on another block's shared memory;
- hardware cluster-wide barriers.

```
__global__ void __cluster_dims__(2, 1, 1) // 2 blocks per cluster
gemm(const half *A, const half *B, float *C);
// or at launch: cudaLaunchKernelEx with cudaLaunchAttributeClusterDimension
```

Portable maximum cluster size is 8 blocks (16 on H100 with an opt-in attribute).

DSMEM in code: a cluster histogram.

```
namespace cg = cooperative_groups;
__global__ void __cluster_dims__(2, 1, 1)
cluster_hist(int *bins, const int *in, size_t n, int bins_per_block) {
  extern __shared__ int smem[]; // this block's slice of bins
  cg::cluster_group cluster = cg::this_cluster();
  for (int i = threadIdx.x; i < bins_per_block; i += blockDim.x) smem[i] = 0;
  cluster.sync(); // all slices zeroed
  for (size_t i = cg::this_grid().thread_rank(); i < n;
       i += gridDim.x * blockDim.x) {
```

```
    int bin = in[i];
    int owner = bin / bins_per_block; // block holding this bin
    int *dst = cluster.map_shared_rank(smem, owner);
    atomicAdd(dst + bin % bins_per_block, 1); // remote smem atomic
  }
  cluster.sync(); // nobody exits while others write
  int *out = bins + cluster.block_rank() * bins_per_block;
  for (int i = threadIdx.x; i < bins_per_block; i += blockDim.x)
    atomicAdd(&out[i], smem[i]);
}
```

Two blocks keep 2× as many bins on chip. Without clusters the atomics go to L2, or each block keeps a full private copy. The final `cluster.sync()` is required: a block's shared memory must stay alive until no other block will touch it. The code assumes bins = 2 × `bins_per_block` and valid input values.

11 Wall 3: size — scaling out and sharing

11.1 NVLink: one all-reduce per training step

Data-parallel training of a 7B model on 8 GPUs sums $7 \times 10^9 \times 2\,\text{B} = 14\,\text{GB}$ of BF16 gradients every step. A ring all-reduce sends and receives

$$2\,\frac{N-1}{N}\,S = 2 \cdot \tfrac{7}{8} \cdot 14\,\text{GB} = 24.5\,\text{GB}$$

per GPU [41], which is bandwidth-optimal. Time at the per-direction link bandwidth:

Link	Bandwidth per direction	Time
PCIe 5.0 x16	≈ 64 GB/s	24.5/64 ≈ 0.38 s
NVLink 4 (H100)	450 GB/s	24.5/450 ≈ 0.054 s

NVSwitch makes the 8 GPUs all-to-all at full NVLink rate. NCCL chooses ring or tree and the route: `ncclAllReduce(grads, grads, n, ncclBfloat16, ncclSum, comm, stream)`.

Pedantic note: what the comparison leaves out

In practice the all-reduce is bucketed and overlapped with the backward pass, so only the non-overlapped part costs time. The PCIe figure assumes GPUs talk peer-to-peer at full link rate; through a shared host root complex it is worse.

11.2 MIG: the opposite problem, jobs too small

Serving BERT-base (≈ 0.2 GB of weights in FP16) on an 80 GB GPU leaves it mostly idle. **Multi-Instance GPU** (Ampere+) partitions one GPU into up to 7 hardware-isolated instances, each with its own SMs, L2 slices, memory controllers and HBM capacity [28]. H100 80GB profiles range from `1g.10gb` ×7 to `7g.80gb`.

Mechanism	How it shares	Isolation
Time slicing	contexts take turns	none (performance), separate address spaces
MPS	kernels of several processes share SMs concurrently	soft
MIG	hardware partitions (SMs, L2, memory)	strong (fault and performance)
vGPU / SR-IOV	virtual GPUs exposed to VMs	VM-level

```
sudo nvidia-smi -i 0 -mig 1 # enable MIG on GPU 0
sudo nvidia-smi mig -i 0 -C -cgi 1g.10gb,1g.10gb,1g.10gb,1g.10gb,1g.10gb,1g.10gb,1g.10gb
nvidia-smi -L # each slice: MIG-<uuid>
CUDA_VISIBLE_DEVICES=MIG-<uuid> python serve.py
```

Every context has its own virtual address space (the GPU has an MMU and page tables). **UVA** gives host and all GPUs one address space, so a pointer names memory uniquely, which enables zero-copy and peer-to-peer access.

11.3 Unified memory and coherence

- **Explicit copies**: pointer-rich structures (trees, graphs, KV caches) must be deep-copied with every pointer rewritten.
- **Managed memory** (`cudaMallocManaged`, demand paging since Pascal): one pointer valid on CPU and GPU; pages migrate on GPU page faults; the working set can exceed HBM. `cudaMemPrefetchAsync` avoids fault storms.
- **Hardware coherence** (Grace Hopper, NVLink-C2C at 900 GB/s): CPU and GPU are cache-coherent; plain `malloc` memory is usable from kernels; ATS lets the GPU use the CPU's page tables [24].

Trajectory: explicit `cudaMemcpy` → managed memory with migration → hardware coherence.

Pedantic note: unified memory costs

A GPU page fault stalls the faulting warps for tens of microseconds while the driver migrates the page; heavy faulting can be far slower than an explicit bulk copy. Unified memory removes programmer effort, not data movement.

12 Synthesis

12.1 One GEMM, five generations

Generation	Wall it hit	Hardware fix	Code idiom
Kepler (2012)	— (the L1 machine)	big RF, shared memory	`__shared__` tiles + FFMA
Volta (2017)	FP32 lane ceiling	Tensor Cores (FP16)	`wmma::mma_sync`
Ampere (2020)	loads hold registers	`cp.async`, TF32, BF16, 2:4 sparsity	`__pipeline_memcpy_async`
Hopper (2022)	address math, sync MMA	TMA, `wgmma`, clusters, FP8	`cp.async.bulk.tensor`
Blackwell (2024)	accumulators fill the RF	TMEM, `tcgen05.mma`, FP4	`tcgen05.mma`

Every row: a real kernel hits a wall, profiling names it, the next chip removes it.

12.2 Roofline, for real

With H100 numbers in (2): ridge = 989/3.35 ≈ 295 FLOP/B (FP16) and 1979/3.35 ≈ 590 FLOP/B (FP8). LLM decode at batch 1 sits far left (about 1 FLOP/B in BF16, 2 in FP8); sparse matrix–vector product is also memory-bound; a large GEMM and FlashAttention-3 sit near the compute roof. TMA, clusters and FP8 push kernels up and to the right.

Pedantic note: "FP8 moves decode to the right"

Halving bytes doubles I for a bandwidth-bound kernel, so decode moves right *and* up along the bandwidth slope. Its roof also rises (FP8 peak), which moves the ridge right, so decode stays memory-bound. The gain is the 2× bandwidth effect, not a change of regime.

12.3 The CPU ↔ GPU dictionary, filled in

Idea	CPU	Modern GPU
hide memory latency	OoO window, caches	many warps + occupancy
specialised compute	SIMD/AVX units	Tensor Cores (mixed precision)
operand delivery	load/store + prefetch	`cp.async` / TMA, shared memory
decoupled access/execute	prefetchers	warp specialisation, async `wgmma`
on-chip sharing	coherent caches	shared memory, cluster DSMEM
scaling out	multi-socket, coherence	NVLink / NVSwitch
partitioning / virtualisation	cores, VMs, SR-IOV	MIG, MPS, vGPU
CPU↔GPU memory	one coherent RAM	unified memory → NVLink-C2C

12.4 Summary: the three walls

Wall	The number	Hopper's answer	Blackwell
Compute	67 vs 989 TFLOPS	Tensor Cores, `wgmma`, FP8	FP4, 2-SM MMA
Feeding	≈300 FLOP/B	`cp.async`, TMA, warp spec., clusters	TMEM
Size	1.1 TB vs 80 GB	NVLink, MIG, NVLink-C2C	72-GPU NVLink domain

The L1 model explains *how* a GPU works; the walls explain *why* it looks the way it does, and where it goes next.

Problems with worked solutions

13 Lecture 1 problems

L1 Poll 1 — What limits the CPU here?

You have 10,000 independent vector adds `c[i] = a[i] + b[i]`. On a big out-of-order CPU, what is the main limit? **(A)** branch prediction **(B)** issue width / number of cores **(C)** the reorder buffer.

Solution. **(B).** The loop is embarrassingly parallel and its single branch is perfectly predictable, so (A) is not a limit. There are no long dependence chains for the ROB to expose, so (C) is not either. Throughput is set by how many adds can execute per cycle: lanes × issue width × cores. The CPU spent its transistors on machinery this workload does not need.

Pedantic note

For large n the true limit is memory bandwidth: each add moves 12 bytes for 1 FLOP ($I = 1/12$ FLOP/B), far left of any roofline ridge. At $n = 10{,}000$ the arrays total 120 KB and fit in L2, so compute-side issue width is the right answer *for the problem as posed*.

L1 Poll 2 — A 50/50 branch

Half the threads in a warp take the `if`, half the `else`. What does the hardware do? **(A)** runs both halves in parallel **(B)** runs them serially with masks **(C)** crashes / undefined.

Solution. **(B).** The warp executes the `if` path with the else-lanes masked, then the `else` path with the if-lanes masked, then reconverges. Time $\approx t_{\text{if}} + t_{\text{else}}$, and each half of the lanes idles for one of the two phases: SIMD efficiency 50%. Minimising intra-warp divergence (e.g. sorting or partitioning work so a warp's 32 threads agree) is a core GPU optimisation.

Pedantic note

On Volta and later the two paths may be *interleaved* rather than run strictly one after the other (independent thread scheduling [18]), but the issue cost is the same. The split also matters only if it falls *within* a warp: if threads 0–15 and 16–31 of every warp disagree, that is divergence; if warp 0 takes `if` and warp 1 takes `else`, there is none.

L1 Design Problem 1 — Fitting warps on an SM

An SM provides 65,536 registers, 164 KB of shared memory, and at most 2048 resident threads (64 warps). Kernel: 256-thread blocks, 48 registers/thread, 16 KB shared memory per block. (a) How many blocks fit per SM, and what limits it? (b) If you cut to 32 registers/thread, does it help?

Solution. (a) Compute the limit from each resource; the minimum binds.

Resource	Blocks
Thread slots	$\lfloor 2048/256 \rfloor = 8$
Registers	$\lfloor 65536/(48 \times 256) \rfloor = \lfloor 5.33 \rfloor = \mathbf{5}$ binds
Shared memory	$\lfloor 164/16 \rfloor = 10$

5 blocks × 8 warps = 40 of 64 warps: occupancy = 62.5%.

(b) With 32 registers: $65536/(32 \times 256) = 8$ blocks, now tied with the thread limit: $8 \times 256 = 2048$ threads = 64/64 warps, **100%** occupancy. Whether it *helps* depends on two things: the compiler must fit the kernel in 32 registers without spilling to local memory (spills add memory traffic), and the kernel must be latency-bound at 62.5% occupancy. If 40 warps already cover the latency, the extra warps gain nothing (see the occupancy note in L1).

Pedantic note

164 KB is the A100 maximum shared memory per SM, and CUDA reserves 1 KB of shared memory per block for system use, so each block really needs 17 KB: $\lfloor 164/17 \rfloor = 9$, still not binding. 48 registers × 32 threads = 1536 per warp, a multiple of the 256-register allocation unit, so no rounding applies. The limit of 32 resident blocks per SM is not reached [25, 19].

L1 Design Problem 2 — Laying out 1 M particles (AoS vs SoA)

1 M particles, each `struct {float x,y,z,vx,vy,vz;}` (24 B). A kernel updates every particle's position (reads `x,y,z`). Why does this array of structs coalesce badly, and how do you fix it?

Solution. With AoS, thread t reads `p[t].x` at byte offset $24t$. One warp's 32 loads of `x` span $32 \times 24 = 768$ bytes: 6 cache lines of 128 B, or 24 sectors of 32 B, to deliver $32 \times 4 = 128$ useful bytes. Sector efficiency $= 128/768 = 1/6$.

Fix: **structure of arrays** (SoA), `float x[N], y[N], z[N], ...`. Then the warp's 32 loads of `x[t]` are 128 consecutive bytes: one line, 4 sectors, 100% used. Same instruction, same math; the layout alone cuts transactions for this load 6×. Intermediate options: AoSoA (structs of short arrays, e.g. 32 particles per tile) keep locality for whole-particle access while coalescing field sweeps.

Pedantic note: the 6× is per load instruction

The kernel reads `x`, `y` and `z`. After the `x` load brings in the 6 lines, the `y` and `z` loads usually hit in L1, so the *DRAM* traffic is 24 B per particle fetched for 12 B needed: 2×, not 6×, provided the lines survive in L1. The 6× applies to a single-field sweep, or to L1-to-register transactions. Either way SoA wins; the size of the win depends on what else is read and on cache behaviour.

L1 Aside — Can `nvcc` coalesce it for you?

Coalescing is done by hardware. Why can't the compiler fix an AoS layout automatically?

Solution. Coalescing is automatic, but only over the addresses *the code emits*: the load/store unit merges a warp's 32 addresses into the fewest 32/128 B transactions. `nvcc` will not transpose data because struct layout is fixed by the C++ ABI; pointers into the array, `memcpy`, host↔device copies and library interop all depend on it. AoS→SoA is therefore a source-level decision. Partial mitigations: per-thread vector loads (`ld.global.v4`) reduce instruction count but stay strided across the warp; caches and `__ldg` recover some reuse but not the transaction count. The real fixes are SoA, AoSoA, or a staging step that loads coalesced into shared memory and redistributes. AoS is fine when each thread uses the whole struct.

14 Lecture 2 problems

L2 Poll 1 — Train Llama 3 70B on the L1 machine

Run the Llama 3 70B training job on the L1 machine (FP32 SIMT lanes only, perfect occupancy, perfect coalescing). What happens? **(A)** about the same **(B)** ∼ 2× longer **(C)** ∼ 15× longer **(D)** it doesn't even fit.

Solution. **(C) and (D).** *Time:* at the same 28% utilisation the sustained rate falls from $0.276 \times 989 = 273$ to $0.276 \times 67 = 18.5$ TFLOPS per GPU. GPU-time $= 6.3 \times 10^{24}/18.5 \times 10^{12} \approx 3.4 \times 10^{11}$ s $\approx 9.4 \times 10^{7}$ GPU-hours ≈ 10,800 GPU-years, 14.8× the real 730. *Fit:* model state is

$16 \times 70 \times 10^9 = 1.12\,\text{TB}$ against 80 GB per GPU, plus activations. A single device cannot hold it, and without fast GPU–GPU links distributing it is communication-bound (Wall 3). Wall 2 also bites: the FP32 machine's ridge point is only $67/3.35 = 20\,\text{FLOP/B}$, so it is compute-starved rather than bandwidth-starved, which is why Wall 1 dominates there.

L2 Poll 2 — Why does FP8 need scaling?

FP8 has very few bits. Why must tensors be scaled before casting to FP8? **(A)** to make them positive **(B)** to fit values into the small representable range **(C)** to coalesce memory.

Solution. **(B).** E4M3 has 4 exponent bits: largest finite value 448, smallest subnormal $2^{-9} \approx 0.002$. Real activations and especially gradients span many more binades, so unscaled they overflow or flush to zero. A per-tensor (or per-block) scale shifts the distribution so its maximum maps near 448. Transformer Engine tracks a history of *amax* ($\max |x|$) and updates the scale each step [29, 38]. (A) is wrong because FP8 has a sign bit; (C) is unrelated to number format.

L2 Poll 3 — Predict Blackwell

On Hopper a consumer warpgroup needs ≈ 232 registers per thread, mostly for the `wgmma` accumulators, and 128 threads cooperate to issue each MMA. Which wall is next, and what would you build?

Solution. The register file has become the feeding bottleneck: accumulators occupy it, limiting tile size and pipeline depth, and issuing an MMA ties up a whole warpgroup. Blackwell's answer [23, 26]:

- **Tensor Memory (TMEM)**: a dedicated 256 KB per SM for MMA accumulators, so the register file is freed again;
- `tcgen05.mma`: an MMA issued by a *single thread*, operands from shared memory/TMEM, result in TMEM;
- **2-SM (CTA-pair) MMA**: two SMs cooperate on one larger MMA, halving per-SM operand traffic;
- **FP4** formats for another doubling of compute (Wall 1).

Walls addressed: feeding (register file) and compute (FP4).

L2 Design Problem 1 — Overlapping load and compute

A kernel loops over K tiles; each iteration loads a tile (L cycles) then computes on it (C cycles), serially. Redesign with `cp.async`/TMA. How many buffers do you need, and what bounds the speedup?

Solution. *Serial:* $T_{\text{serial}} = K(L + C)$.

Double-buffered: while computing tile k from buffer k mod 2, load tile $k{+}1$ into the other buffer. Two buffers suffice to overlap one load with one compute. Exact total:

$$T_{\text{db}} = L + (K - 1)\max(L, C) + C,$$

which for large K is $\approx K \max(L, C)$. (The slide's $L + K\max(L, C)$ is the same to first order.) Speedup:

$$\frac{T_{\text{serial}}}{T_{\text{db}}} \xrightarrow{K\to\infty} \frac{L + C}{\max(L, C)} \le 2,$$

with equality when $L = C$. You can never beat the slower stage, so size the tile to balance L and C.

Pedantic note: when more than 2 buffers help

L mixes latency and transfer time: $L = \lambda + \beta$, where λ is memory latency and β = bytes/bandwidth. With 2 buffers a load must *complete* within one compute, needing $\lambda + \beta \leq C$. With S stages, $S - 1$ loads are in flight, so the condition becomes roughly $\beta \leq C$ and $\lambda \leq (S-1)C$: extra stages hide latency (Little's law again) but cannot beat bandwidth. That is why Ampere/Hopper kernels use 3–5 stages, and why freeing registers (`cp.async`) and threads (TMA) to afford them mattered.

L2 Design Problem 2 — Choosing precision for a transformer

You train a transformer whose cost is dominated by GEMMs (QKV, FFN). Pick a precision for the matmul inputs and for accumulation. Where must you stay FP32? Where does FP8 need scaling?

Solution.

Quantity	Precision
GEMM inputs (weights, activations)	BF16; or FP8 (E4M3 forward, E5M2 gradients)
Tensor Core accumulation	FP32
Master weights, optimiser state (m, v)	FP32
Softmax, LayerNorm, loss, reductions	FP32 (or computed in FP32 internally)

Reasons. BF16 keeps FP32's 8-bit exponent, so it needs no loss scaling; FP16 has only 5 exponent bits and needs dynamic loss scaling to keep small gradients from underflowing [36, 37]. FP8's range is tiny, so each tensor needs a scale factor (Poll 2); E5M2's wider range suits gradients and E4M3's extra mantissa bit suits forward values [38]. Accumulations and weight updates stay FP32 because small increments added to large sums vanish in low precision: in BF16 (8 significant bits), $1 + 2^{-9}$ rounds to 1. Rule: low precision where *throughput* lives, FP32 where *accuracy accumulates*, and scale.

Further reading by topic

Topic	Read
ILP limits, power wall	[1] Ch. 3–4, [8], [10]
Throughput design, SIMT	[2], [5], [6], [4] Ch. 2–3
Divergence	[7], [4] Ch. 3, [18]
Occupancy vs ILP	[12], [25] (occupancy)
CUDA programming, tiling	[3] Part I, [25]
Roofline	[13]
Microarchitecture measurements	[16], [17], [15]
Hopper features	[20], [25], [26], [27]
Kernels using them	[35], [39], [40]
Low precision	[36], [37], [38], [29]
Workload numbers	[30], [31], [32], [33], [34]
Scale-out and sharing	[41], [28], [24]

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017. Ch. 4: Data-level parallelism in vector, SIMD, and GPU architectures.

[2] **K. Fatahalian and M. Houston, "A closer look at GPUs," *Communications of the ACM*, 51(10):50–57, 2008. doi:10.1145/1400181.1400197**

[3] W.-m. W. Hwu, D. B. Kirk and I. El Hajj, *Programming Massively Parallel Processors: A Hands-on Approach*, 4th ed. Morgan Kaufmann, 2022.

[4] T. M. Aamodt, W. W. L. Fung and T. G. Rogers, *General-Purpose Graphics Processor Architectures*. Synthesis Lectures on Computer Architecture, Morgan & Claypool, 2018.

[5] **E. Lindholm, J. Nickolls, S. Oberman and J. Montrym, "NVIDIA Tesla: A unified graphics and computing architecture," *IEEE Micro*, 28(2):39–55, 2008. doi:10.1109/MM.2008.31**

[6] J. Nickolls, I. Buck, M. Garland and K. Skadron, "Scalable parallel programming with CUDA," *ACM Queue*, 6(2):40–53, 2008.

[7] W. W. L. Fung, I. Sham, G. Yuan and T. M. Aamodt, "Dynamic warp formation and scheduling for efficient GPU control flow," in *Proc. MICRO-40*, 2007.

[8] D. W. Wall, "Limits of instruction-level parallelism," in *Proc. ASPLOS-IV*, 1991.

[9] R. H. Dennard et al., "Design of ion-implanted MOSFETs with very small physical dimensions," *IEEE J. Solid-State Circuits*, 9(5):256–268, 1974.

[10] H. Esmaeilzadeh, E. Blem, R. St. Amant, K. Sankaralingam and D. Burger, "Dark silicon and the end of multicore scaling," in *Proc. ISCA*, 2011.

[11] J. D. C. Little, "A proof for the queuing formula $L = \lambda W$," *Operations Research*, 9(3):383–387, 1961.

[12] **V. Volkov, "Better performance at lower occupancy," NVIDIA GPU Technology Conference (GTC), 2010.** `https://www.nvidia.com/content/GTC-2010/pdfs/2238_GTC2010.pdf`

[13] **S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, 52(4):65–76, 2009. doi:10.1145/1498765.1498785. Free tech-report version:** `https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.pdf`

[14] J. E. Smith, "Decoupled access/execute computer architectures," in *Proc. ISCA*, 1982.

[15] Z. Jin et al., "Uncovering real GPU NoC characteristics: Implications on interconnect architecture," in *Proc. MICRO-57*, 2024.

[16] Z. Jia, M. Maggioni, B. Staiger and D. P. Scarpazza, "Dissecting the NVIDIA Volta GPU architecture via microbenchmarking," arXiv:1804.06826, 2018. `https://arxiv.org/abs/1804.06826`

[17] W. Luo, R. Fan, Z. Li, D. Du, Q. Wang and X. Chu, "Benchmarking and dissecting the Nvidia Hopper GPU architecture," in *Proc. IPDPS*, 2024. arXiv:2402.13499. `https://arxiv.org/abs/2402.13499`

[18] NVIDIA, *NVIDIA Tesla V100 GPU Architecture* (whitepaper), 2017. Section on independent thread scheduling.

[19] NVIDIA, *NVIDIA A100 Tensor Core GPU Architecture* (whitepaper), 2020.

[20] NVIDIA, *NVIDIA H100 Tensor Core GPU Architecture* (whitepaper), 2022. `https://resources.nvidia.com/en-us-hopper-architecture/nvidia-h100-tensor-c`

[21] NVIDIA, *H100 Tensor Core GPU* datasheet. https://www.nvidia.com/en-us/data-center/h100/

[22] NVIDIA, *RTX A6000* datasheet.

[23] NVIDIA, *NVIDIA Blackwell Architecture Technical Brief*, 2024.

[24] NVIDIA, *NVIDIA Grace Hopper Superchip Architecture* (whitepaper), 2023.

[25] NVIDIA, *CUDA C++ Programming Guide*. Sections: hardware implementation, asynchronous data copies, `cuda::pipeline`, Tensor Memory Access, thread-block clusters, distributed shared memory, L2 access management. https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[26] NVIDIA, *Parallel Thread Execution ISA*. Instructions `mma`, `wgmma.mma_async`, `setmaxnreg`, `cp.async`, `cp.async.bulk.tensor`, `tcgen05`. https://docs.nvidia.com/cuda/parallel-thread-execution/

[27] NVIDIA, *CUTLASS: CUDA Templates for Linear Algebra Subroutines*. https://github.com/NVIDIA/cutlass

[28] NVIDIA, *Multi-Instance GPU User Guide*. https://docs.nvidia.com/datacenter/tesla/mig-user-guide/

[29] NVIDIA, *Transformer Engine* documentation. https://docs.nvidia.com/deeplearning/transformer-engine/

[30] J. Kaplan et al., "Scaling laws for neural language models," arXiv:2001.08361, 2020. https://arxiv.org/abs/2001.08361

[31] Meta, *Llama 3 model card*, 2024. https://huggingface.co/meta-llama/Meta-Llama-3-70B

[32] A. Grattafiori, A. Dubey et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024. https://arxiv.org/abs/2407.21783

[33] D. Narayanan et al., "Efficient large-scale language model training on GPU clusters using Megatron-LM," in *Proc. SC'21*, 2021. arXiv:2104.04473.

[34] S. Rajbhandari, J. Rasley, O. Ruwase and Y. He, "ZeRO: Memory optimizations toward training trillion parameter models," in *Proc. SC'20*, 2020. arXiv:1910.02054.

[35] S. Boehm, "How to optimize a CUDA matmul kernel for cuBLAS-like performance: a worklog," 2022. https://siboehm.com/articles/22/CUDA-MMM

[36] P. Micikevicius et al., "Mixed precision training," in *Proc. ICLR*, 2018. arXiv:1710.03740.

[37] D. Kalamkar et al., "A study of BFLOAT16 for deep learning training," arXiv:1905.12322, 2019.

[38] P. Micikevicius et al., "FP8 formats for deep learning," arXiv:2209.05433, 2022.

[39] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. ICLR*, 2024. arXiv:2307.08691.

[40] **J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Proc. NeurIPS*, 2024. https://arxiv.org/abs/2407.08608**

[41] P. Patarasuk and X. Yuan, "Bandwidth optimal all-reduce algorithms for clusters of workstations," *Journal of Parallel and Distributed Computing*, 69(2):117–124, 2009.